Controlled Automation Failure Injection

Laurence Matthew B Manuel | BSIT 3A | PFEBSIT 002 | Project 29

Prepared 8 November 2026 | Asia/Manila | Week 13 SA7

Task and predicted safe behavior

Activity 2 tests duplicate processing, endless retries and silent data loss. Before execution, scenarios.md predicts one receipt for successful work, one recovered receipt after a transient retry, immediate manual review for permanent failure, and manual review after exactly three persistent-timeout attempts. Invalid and unauthorized requests must not mutate workflow state. A simulated process crash after receipt insertion must roll back receipt and attempt together, leaving the event available for recovery.

Method and controlled changes

Local-only week13:evidence adds four fictional progress-summary events with success, transient, permanent and timeout injection modes. Injection is not exposed through a public API. It uses the configured retry clock and waits in real time, without resetting attempts or changing the host clock. PHPUnit separately uses controlled test-clock advancement to verify due rules and early retry suppression. Expected and actual results, source build label, real capture timestamps and correlations remain in JSON/JUnit evidence.

Observed real time result

The run evidence:20261108100635 completed in 190.52 seconds. Success: delivered, 1 attempt. Transient: delivered, 2 attempts. Permanent: manual_review, 1 attempt. Persistent timeout: manual_review, 3 attempts. There were seven attempts, two receipts, two manual-review records and zero pending events. No real email/SMS was sent. The final capture was 2026-11-08T18:09:45+08:00.

Prevention controls and alternatives

Unique request/event keys and one receipt per outbox ID prevent duplicate simulated processing. Status and available-at checkpoints prevent premature retry; a three-attempt ceiling prevents endless retry; retained manual-review rows prevent failures from disappearing. A transactional receiver gives a stronger local recovery guarantee than an in-memory send-then-log callback. A real external receiver would need provider deduplication or reconciliation because an SQL rollback cannot undo an external message.

Improvement and rerun

An initial backend fixture failed because clock travel expired its session, correctly producing 401. The fixture now renews only controlled session expiry after changing test time. The failed-case rerun and regression suite passed; no access rule was removed. Initial Flutter analysis/test failures were also retained and corrected. Stale reminders are superseded after cancellation, and a crash test confirms a retained pending event can produce exactly one receipt on rerun. Consult final logs for the latest suite counts.

Activity Two Interpretation

Laurence Matthew B Manuel | BSIT 3A | PFEBSIT 002 | Project 29

Prepared 8 November 2026 | Asia/Manila | Week 13 SA7

Observed facts

The real-time timeline shows a transient event waiting before its second attempt and then reaching delivered. Permanent and exhausted events remain visible as manual_review. The API tests reject student attempts to close a manual-review item and allow the coordinator to record a handling decision without sending again. A separate transaction test deliberately throws after inserting a simulated receipt; the receipt and attempt roll back, and a clean rerun succeeds. Duplicate scan/worker assertions retain one event and receipt.

Reasoned conclusions

The combination of durable checkpoints, uniqueness constraints, row locking and transaction boundaries supports safe retry for this controlled receiver. Bounded backoff and an explicit manual owner make failure actionable. This evidence is stronger than a single success screenshot, but remains limited to small fixtures on one host. It does not prove external delivery, production concurrency, unlimited outage recovery or a complete distributed circuit breaker.

Recommendations and operational authority

The coordinator should review every manual-review row, reconcile the fictional recipient and record a follow-up decision. Operators should not delete failures or reset attempts to make a demo appear green. Before a real provider is introduced, test provider-side idempotency, network timeout and ambiguous-response reconciliation. Measure pending age and alert through a channel independent of the failed transport. Evaluate scanner memory and throughput on bounded ordinary and peak workloads in the later authorized milestone.

Evidence and remaining work

Use logs/controlled-delivery.json, tests/phpunit-results.xml, backend-tests.txt and the source manifest. Run php artisan week13:evidence to repeat a real-time test; run php artisan test for regression and crash checks. These records contain fictional identifiers and no credentials. Real source and running Flutter screenshots are indexed with actual capture times and hashes. The student confirmed the reflection and project registration on 8 November 2026. The actual oral assessment, marks and any instructor-prescribed acknowledgment wording remain external actions.